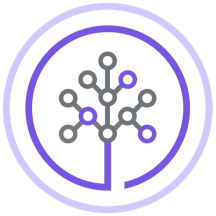
Lab: Build an MCP Server



Skills Network

Estimated time: 30 minutes

Introduction

You worked with the California culinary map dataset earlier in this capstone project. In this lab, you will use the same dataset to build an MCP server and create relevant tools. This server will later be accessed by clients in upcoming labs, allowing them to connect and query the data using the MCP protocol.

Learning Objectives

By the end of this lab, you will have:

- Created an **MCP resource** that exposes raw text data
- Implemented three **MCP tools** for searching and retrieving restaurant information
- Run a **FastMCP server** locally and tested it with basic commands

Set up the environment

Let's create a virtual environment to keep dependencies isolated.

bash

```
pip install virtualenv
virtualenv .venv
source .venv/bin/activate
```

Run

Install the required library:

bash

```
pip install fastmcp==3.1.0
```

Run

Download the data files needed for this lab:

bash

```
# Unstructured California culinary map
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/_n

# Structured restaurant data (JSON)
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/lx

# Augmented user reviews (JSON)
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/oM
```

Run

Create the server file:

bash

```
touch server.py
```

Run

Open server.py in IDE

MCP Resources

Imports and server initialization

At the top of `server.py`, import the required libraries and initialize the FastMCP server instance.

```python
# Libraries to import to create our MCP server and handle data loading
from fastmcp import FastMCP
from pathlib import Path
import json

# Initializing our MCP server instance
mcp = FastMCP("Connoisseur-Server")
```

`FastMCP` is the main class from the `fastmcp` library. Giving it a name (`"Connoisseur-Server"`) identifies this server when clients connect to it.

Data paths and helper functions

Next, define the file paths and two helper functions that load the JSON data files.

```python
# Data paths
DATA_DIR = Path(__file__).parent
CULINARY_MAP_PATH = DATA_DIR / "California-Culinary-Map.txt"
RESTAURANT_DATA_PATH = DATA_DIR / "structured-restaurant-data.json"
REVIEW_DATA_PATH = DATA_DIR / "augmented-user-review.json"

# Helper functions
def load_restaurant_data() -> list[dict]:
    """Load the structured restaurant data produced in Module 1."""
    with open(RESTAURANT_DATA_PATH, "r") as f:
        return json.load(f)

def load_review_data() -> list[dict]:
    """Load the augmented user reviews produced in Module 1."""
    with open(REVIEW_DATA_PATH, "r") as f:
        return json.load(f)
```

`Path(__file__).parent` resolves to the directory containing `server.py`, so the data files are always found relative to the script regardless of where you run it from.

Defining the resource

Next, create an **MCP resource** that exposes static or semi-static data that a client can read. Here, the full California Culinary Map text is exposed.

```python
# MCP Resource - Exposing the Raw Culinary Map data
@mcp.resource("culinary-map://california")
def get_culinary_map() -> str:
    """The full raw California Culinary Map text from Module 1.
    Contains detailed descriptions of 100+ restaurants across California
    including their vibes, cuisines, ratings, and price ranges."""
    return CULINARY_MAP_PATH.read_text()
```

The string `"culinary-map://california"` is the **URI** a client uses to request this resource. The decorator registers the function so FastMCP serves its return value at that URI.

Set Up MCP Tools

You are still working on `server.py`, so continue adding the following code to it.

MCP tools are callable functions that a client (or agent) can invoke with arguments. Unlike resources, tools accept input and perform logic before returning a result.

Tool 1 — Get Restaurant Info

This tool searches the structured JSON data for a restaurant by name.

```python
# TOOL 1 — Get Restaurant Info (Structured Search)
@mcp.tool()
def get_restaurant_info(restaurant_name: str) -> str:
    """Search for a restaurant by name and return its structured details
    including cuisine, rating, price range, and signature dish."""
    restaurants = load_restaurant_data()
    query = restaurant_name.lower().strip()

    # Finding restuarants that match the query in the structured JSON data
    matches = []
    for restaurant in restaurants:
        name = restaurant["name"].lower()
        if query in name or name in query:
            matches.append(restaurant)

    # Return a not found message if no matches are found
    if not matches:
        return json.dumps(
            {
                "status": "not_found",
                "message": f"No restaurant found matching '{restaurant_name}'
                "suggestion": "Try a partial name like 'Iron' or 'Sakura'.",
            },
            indent=2,
        )

    return json.dumps(
        {"status": "found", "count": len(matches), "results": matches},
        indent=2,
    )
```

The tool does a **partial name match** in both directions (`query in name` and `name in query`) so searches such as `"Iron"` still find `"Iron Chef Kitchen"`. Results are serialized as JSON strings — the standard way MCP tools return structured data.

Tool 2 — Recommend by Vibe

This tool performs a two-pass search: first against structured vibe tags in the JSON, then against the raw text of the culinary map.

python

```python
# TOOL 2 — Recommend by Vibe (Semantic Search)
@mcp.tool()
def recommend_by_vibe(vibe: str) -> str:
    """Find restaurants that match a given vibe or atmosphere keyword.
    Searches both structured vibe tags and raw text descriptions.
    Examples of vibe keywords: "moody", "sun-drenched", "romantic"""
    restaurants = load_restaurant_data()
    vibe_lower = vibe.lower().strip()

    # Pass 1: Search structured vibe tags in JSON
    structured_matches = []
    for restaurant in restaurants:
        vibes_list = [v.lower() for v in restaurant.get("vibes", [])]
        description = restaurant.get("description", "").lower()

        if any(vibe_lower in v for v in vibes_list) or vibe_lower in descript
            structured_matches.append(
                {
                    "name": restaurant["name"],
                    "neighborhood": restaurant["neighborhood"],
                    "cuisine": restaurant["cuisine"],
                    "rating": restaurant["rating"],
                    "vibes": restaurant["vibes"],
                    "price_range": restaurant["price_range"],
                }
            )

    # Pass 2: Search the raw text for additional matches
    raw_text = CULINARY_MAP_PATH.read_text()
    paragraphs = raw_text.split("\n\n")
    text_excerpts = []
    for para in paragraphs:
        if vibe_lower in para.lower() and para.strip():
            text_excerpts.append(para.strip()[:300])

    return json.dumps(
        {
            "vibe_searched": vibe,
            "structured_matches": structured_matches,
            "raw_text_excerpts": text_excerpts[:5],
        },
        indent=2,
    )
```

The two-pass approach ensures nothing is missed: structured data gives precise matches while the raw text captures descriptive language that wasn't tagged as a formal vibe.

Tool 3 — Get Review

This tool retrieves the full review record for a named restaurant.

Note: in this section you will need to write the not found message if no review matches the query passed to the tool. Locate the lines that are marked as `TODO` and replace the `...` in the code with your own message.

Hint: look to see how this is implemented for tool 1

python

```python
# TOOL 3 — Get Review (Returns Review Data for Lab 2 Demonstration)
@mcp.tool()
def get_review(restaurant_name: str) -> str:
    """Retrieve the full review for a restaurant."""
    reviews = load_review_data()
    query = restaurant_name.lower().strip()

    # Find the matching review
    matching_review = None
    for review in reviews:
        if query in review["restaurant_name"].lower():
            matching_review = review
            break

    # Return a not found message if no review matches the query
    if not matching_review:
        return json.dumps(
            {
                "status": ... , #TODO
                "message": ... , #TODO
            },
            indent=2,
        )

    return json.dumps(
        {
            "status": "found",
            "restaurant": matching_review["restaurant_name"],
            "reviewer": matching_review["reviewer"],
            "rating": matching_review["rating"],
            "review_text": matching_review["review_text"],
            "image_description": matching_review.get("image_description", "N/
            "visit_date": matching_review.get("visit_date", "N/A"),
        },
        indent=2,
    )
```

This tool will be particularly useful in later labs when an agent needs to pull up a specific review to analyze or summarize.

Entry Point

Finally, add the entry point so the server starts when the script is run directly.

python

```python
# Run the Server
if __name__ == "__main__":
    mcp.run()
```

Run the MCP Server

Create `test.py` to test whether our MCP server is working:

bash

```
touch test.py
```

Run

Open test.py in IDE

Copy the following into test.py

python

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def run_test():

    server_params = StdioServerParameters(
        command="python3",
        args=["server.py"],
    )
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            result = await session.call_tool("get_restaurant_info", arguments

            print("\n--- START SCREENSHOT ---")
            print(result.content[0].text)
            print("--- END SCREENSHOT ---\n")

if __name__ == "__main__":
    asyncio.run(run_test())
```

Run test.py

```bash
python test.py
```

Run

If this command runs without error and you get a JSON output in your terminal, that means our MCP server is working! The terminal output should look something like this:

```ruby
--- START SCREENSHOT ---
{
  "status": ...
  "count": ...
  "results": [
    {
      "name": ...
      "neighborhood": ...
      "cuisine": ...
      "type": ...
      "rating": ...
      "price_range": ...
      "signature_dish": ...
      "vibes": ...
      "description": ...
    }
  ]
}
--- END SCREENSHOT ---
```

Take a screenshot of the section in your terminal output from `START SCREENSHOT` to `END SCREENSHOT` and name it `M4L1_Configure_Tools_Data_MCP_Server.jpg`. This screenshot will be used as one of the submission components later for your Final Project.

Conclusion

You have built a complete **FastMCP server** that exposes California restaurant data over the Model Context Protocol. You can now effectively configure data and tools in an MCP server!

By completing this lab, you have:

- Defined an **MCP resource** to serve raw text data at a named URL
- Written three **MCP tools** covering exact name lookup, vibe-based search, and review retrieval
- Run the server locally and tested it with CLI commands

This server is the data layer that the upcoming labs will connect to — an agent can now discover these tools automatically and call them to answer questions about California restaurants.

Author(s)

Abdul Fatir | Data Scientist @ IBM

MCP Resources

Imports and server initialization

At the top of `server.py`, import the required libraries and initialize the FastMCP server instance.

```python
# Libraries to import to create our MCP server and handle data loading
from fastmcp import FastMCP
from pathlib import Path
import json

# Initializing our MCP server instance
mcp = FastMCP("Connoisseur-Server")
```

`FastMCP` is the main class from the `fastmcp` library. Giving it a name (`"Connoisseur-Server"`) identifies this server when clients connect to it.

Data paths and helper functions

Next, define the file paths and two helper functions that load the JSON data files.

```python
# Data paths
DATA_DIR = Path(__file__).parent
CULINARY_MAP_PATH = DATA_DIR / "California-Culinary-Map.txt"
RESTAURANT_DATA_PATH = DATA_DIR / "structured-restaurant-data.json"
REVIEW_DATA_PATH = DATA_DIR / "augmented-user-review.json"

# Helper functions
def load_restaurant_data() -> list[dict]:
    """Load the structured restaurant data produced in Module 1."""
    with open(RESTAURANT_DATA_PATH, "r") as f:
        return json.load(f)

def load_review_data() -> list[dict]:
    """Load the augmented user reviews produced in Module 1."""
    with open(REVIEW_DATA_PATH, "r") as f:
        return json.load(f)
```

`Path(__file__).parent` resolves to the directory containing `server.py`, so the data files are always found relative to the script regardless of where you run it from.

Defining the resource

Next, create an **MCP resource** that exposes static or semi-static data that a client can read. Here, the full California Culinary Map text is exposed.

```python
# MCP Resource - Exposing the Raw Culinary Map data
@mcp.resource("culinary-map://california")
def get_culinary_map() -> str:
    """The full raw California Culinary Map text from Module 1.
    Contains detailed descriptions of 100+ restaurants across California
    including their vibes, cuisines, ratings, and price ranges."""
    return CULINARY_MAP_PATH.read_text()
```

The string `"culinary-map://california"` is the **URI** a client uses to request this resource. The decorator registers the function so FastMCP serves its return value at that URI.

← Set up the environment

Set Up MCP Tools →